

TP Séance 3

Contexte

À la fin de ce TP, vous aurez :

- Installé **Kind** (*Kubernetes IN Docker*) et créé votre premier cluster Kubernetes local.
- Installé **kubectl**, l'outil en ligne de commande pour piloter un cluster K8s.
- Créé un **namespace dédié** `anfa` pour isoler les ressources du projet.
- Déployé **MinIO sur Kubernetes** via 3 manifestes YAML : PersistentVolumeClaim, Deployment, Service.
- Observé concrètement le **self-healing** : supprimé un pod et vu Kubernetes le recréer automatiquement.
- **Scalé un Deployment** de 1 à 3 replicas en une commande.
- Activé un **Ingress Controller** et compris son rôle (sans configurer d'Ingress complet).

Partie 0 : Préparer la branche de la séance

Depuis la racine de votre fork local :

```
# Récupérer les éventuelles mises à jour du tronc
git checkout main
git pull

# Créer la branche de la séance 3
git checkout -b seance-03

# Créer le dossier de travail et son sous-dossier manifests/
mkdir -p seance-03/manifests
cd seance-03
```

Point de vérification 0. Vous êtes sur la branche `seance-03`, dans `seance-03/`, et le sous-dossier `manifests/` existe.

Partie 1 : Installer Kind et kubectl

1.1 Installer Kind

Linux :

```
# Télécharger le binaire (architecture amd64)
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.32.0/kind-linux-amd64

# Le rendre exécutable et déplacer dans un dossier du PATH
chmod +x ./kind && sudo mv ./kind /usr/local/bin/kind
```

Windows (avec winget) :

```
winget install Kubernetes.kind
```

Vérifier l'installation :

```
kind version
```

Résultat attendu : `kind v0.32.0 ...` (ou version équivalente il faut **au moins v0.32.0** pour disposer de l'image Kubernetes v1.35.1 utilisée plus bas).

1.2 Installer kubectl

`kubectl` est l'outil en ligne de commande qui parle à l'API Server d'un cluster Kubernetes. C'est votre interface principale avec K8s.

Linux :

```
# Télécharger la dernière version stable
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

chmod +x kubectl
sudo mv kubectl /usr/local/bin/
```

Windows :

```
winget install Kubernetes.kubectl
```

Vérifier l'installation :

```
kubectl version --client
```

Résultat attendu : affichage de la version client de kubectl. Aucun serveur n'est encore en place c'est normal.

Point de vérification 1. `kind version` et `kubectl version --client` renvoient tous deux des numéros de version.

Partie 2 : Créer un cluster Kubernetes avec Kind

2.1 Créer le cluster

```
# Crée un cluster Kubernetes nommé "anfa" dans des conteneurs Docker
# --image épingle la version de Kubernetes (sinon kind prend sa version par défaut)
kind create cluster --name anfa --image kindest/node:v1.35.1
```

Résultat attendu :

```
Creating cluster "anfa" ...
 ✓ Ensuring node image (kindest/node:v1.35.1)
 ✓ Preparing nodes
 ✓ Writing configuration
 ✓ Starting control-plane
 ✓ Installing CNI
 ✓ Installing StorageClass
 ✓ Waiting ≤ 50s for control-plane = Ready
Set kubectl context to "kind-anfa"
You can now use your cluster with:

kubectl cluster-info --context kind-anfa
```

La création prend environ 30 secondes à 1 minute. C'est beaucoup plus rapide que Minikube ou une VM classique.

2.2 Moment-clé : observer ce que Kind a vraiment fait

```
# Lister les conteneurs Docker en cours d'exécution
docker ps
```

Vous voyez quelque chose comme :

CONTAINER ID	IMAGE	COMMAND	...	NAMES
abc123def	kindest/node:v1.35.1	"/usr/local/bin/entr..."	...	anfa-control-plane

À comprendre. Le nœud Kubernetes est un conteneur Docker. Kind a lancé un conteneur Docker basé sur l'image `kindest/node`, qui contient à la fois le Control Plane et le Worker (par défaut, un seul nœud sert les deux rôles).

C'est une démonstration spectaculaire : Kubernetes orchestre des conteneurs, mais Kubernetes lui-même peut tourner **dans** des conteneurs. La conteneurisation est partout.

2.3 Vérifier que kubectl parle au cluster

```
# Affiche les informations du cluster auquel kubectl est connecté
kubectl cluster-info
```

Résultat attendu :

```
Kubernetes control plane is running at https://127.0.0.1:XXXXX
CoreDNS is running at https://127.0.0.1:XXXXX/api/v1/namespaces/kube-system/services/...
```

```
# Liste les nœuds du cluster
kubectl get nodes
```

Résultat attendu :

NAME	STATUS	ROLES	AGE	VERSION
anfa-control-plane	Ready	control-plane	1m	v1.35.1

Point de vérification 2. `kubectl get nodes` affiche un nœud avec le statut `Ready`. Vous avez un cluster Kubernetes fonctionnel sur votre machine.

Partie 3 : Premier contact avec kubectl

Avant de déployer Anfa, prenons 5 minutes pour explorer ce qui tourne déjà dans le cluster.

3.1 Voir tout ce qui tourne dans le cluster

```
# --all-namespaces : affiche les pods de tous les namespaces, pas seulement "default"
kubectl get pods --all-namespaces
```

Résultat attendu :

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-...	1/1	Running	0	2m
kube-system	etcd-anfa-control-plane	1/1	Running	0	2m
kube-system	kindnet-...	1/1	Running	0	2m
kube-system	kube-apiserver-anfa-control-plane	1/1	Running	0	2m
kube-system	kube-controller-manager-anfa-control-plane	1/1	Running	0	2m
kube-system	kube-proxy-...	1/1	Running	0	2m
kube-system	kube-scheduler-anfa-control-plane	1/1	Running	0	2m
local-path-storage	local-path-provisioner-...	1/1	Running	0	2m

À comprendre. Tous les composants du Control Plane qu'on a vus en CM (API Server, etcd, Scheduler, Controller Manager) sont eux-mêmes... **des pods Kubernetes** qui tournent dans le namespace `kube-system`. C'est l'élégance de K8s : il se gère lui-même.

3.2 La documentation intégrée

```
# Explique ce qu'est un Pod et ses champs
kubectl explain pod

# Plus profond : explique le champ "spec" d'un Pod
kubectl explain pod.spec

# Encore plus : les conteneurs dans un pod
kubectl explain pod.spec.containers
```

À retenir. `kubectl explain` est la documentation officielle de chaque objet K8s, directement dans votre terminal. Indispensable quand on écrit un manifeste.

Partie 4 : Créer le namespace `anfa`

4.1 Créer le namespace

```
# Crée un namespace nommé "anfa"  
kubectl create namespace anfa
```

Résultat attendu : `namespace/anfa created`.

4.2 Configurer kubectl pour utiliser ce namespace par défaut

Pour ne pas avoir à taper `-n anfa` à chaque commande :

```
# Définit "anfa" comme namespace par défaut dans le contexte courant  
kubectl config set-context --current --namespace=anfa
```

Vérifier :

```
# Doit être vide : aucun pod dans le namespace anfa pour le moment  
kubectl get pods
```

Point de vérification 4. Le namespace `anfa` existe (`kubectl get namespaces` le montre), et `kubectl get pods` ne retourne aucun pod (le namespace est vide).

Partie 5 : Déployer MinIO sur Kubernetes

Nous allons déployer MinIO avec **3 manifestes YAML** appliqués dans l'ordre :

1. **PersistentVolumeClaim** : demande de stockage persistant.
2. **Deployment** : description du pod MinIO et de son cycle de vie.
3. **Service** : exposition du pod sur le réseau.

5.1 Manifeste 1 : Le PersistentVolumeClaim

Créez le fichier `seance-03/manifests/minio-pvc.yaml` :

```
# minio-pvc.yaml
# -----
# Demande de stockage persistant pour MinIO.
# Équivalent du volume nommé "anfa-minio-data" en Docker Compose.

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: minio-pvc          # nom de la ressource (référéncé par le Deployment)
spec:
  # accessModes : qui peut monter ce volume
  #   ReadWriteOnce : un seul nœud à la fois en lecture/écriture (suffisant ici)
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi          # demande 2 Go de stockage
```

Appliquer le manifeste :

```
# kubectl apply : crée ou met à jour la ressource décrite dans le fichier
# -f : "file" (chemin du manifeste)
kubectl apply -f manifests/minio-pvc.yaml
```

Résultat attendu : `persistentvolumeclaim/minio-pvc created.`

Vérifier :

```
kubectl get pvc
```

Résultat attendu :

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
minio-pvc	Bound	pvc-xxxx...	2Gi	RW0	standard	10s

Le statut `Bound` signifie que K8s a trouvé un PersistentVolume pour satisfaire la demande. Kind utilise un **provisioner local** qui crée les volumes automatiquement.

Point de vérification 5.1. Le PVC `minio-pvc` est en statut `Bound` avec 2 Gi de capacité.

5.2 Manifeste 2 : Le Deployment

Créez le fichier `seance-03/manifests/minio-deployment.yaml`.

Utilisez le manifeste disponible ici : <https://gist.github.com/denisakp/70bd5d188eaefacf192907d46e06e5ee>

Appliquer :

```
kubectl apply -f manifests/minio-deployment.yaml
```

Suivre le démarrage :

```
kubectl get pods -w
```

Observez le pod passer par les états `Pending` → `ContainerCreating` → `Running`.

Une fois en `Running`, quittez avec Ctrl+C, puis :

```
# Vérifier l'état détaillé
kubectl get deployment minio
kubectl get pods
```

Résultat attendu :

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
minio	1/1	1	1	30s

NAME	READY	STATUS	RESTARTS	AGE
minio-xxxxxxxxxx-yyyyy	1/1	Running	0	30s

Point de vérification 5.2. Le Deployment `minio` affiche `1/1` ready, et le pod est `Running`.

5.3 Voir les logs du pod

```
# Afficher les logs du Deployment (pratique : pas besoin de connaître le nom exact du pod)
kubectl logs deployment/minio
```

Vous voyez les logs de démarrage de MinIO. Cherchez la ligne `API: http://...:9000` qui confirme que MinIO est opérationnel.

5.4 Manifeste 3 : Le Service

Créez le fichier `seance-03/manifests/minio-service.yaml` :

```
# minio-service.yaml
# -----
# Service : adresse réseau stable qui pointe vers les pods MinIO.
# Type NodePort : expose le service sur un port de chaque nœud du cluster
#                → permet l'accès depuis l'extérieur.

apiVersion: v1
kind: Service
metadata:
  name: minio
spec:
  type: NodePort
  selector:
    app: minio                # cible les pods ayant ce label
  ports:
    - name: api
      port: 9000               # port du service (interne au cluster)
      targetPort: 9000         # port du conteneur cible
      nodePort: 30900          # port exposé sur les nœuds (30000-32767)
    - name: console
      port: 9001
      targetPort: 9001
      nodePort: 30901
```


Appliquer :

```
kubectl apply -f manifests/minio-service.yaml
```

Vérifier :

```
kubectl get service minio
```

Résultat attendu :

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
minio	NodePort	10.96.xxx.xxx	<none>	9000:30900/TCP,9001:30901/TCP	15s

Point de vérification 5.3. Le Service `minio` est de type `NodePort` et expose les ports 30900 (API) et 30901 (console).

5.5 Accéder à la console MinIO depuis votre navigateur

Note importante avec Kind. Par défaut, le NodePort de Kind n'est pas accessible depuis l'hôte. Pour y accéder, nous utilisons le **port-forwarding** de kubectl.

Ouvrez **deux nouveaux terminaux** (en plus de votre terminal de travail principal) :

Terminal A :

```
# Redirige le port 9001 local vers le port 9001 du service "minio"
# Laissez cette commande tourner. Ctrl+C pour arrêter.
kubectl port-forward service/minio 9001:9001
```

Terminal B :

```
kubectl port-forward service/minio 9000:9000
```

Dans votre navigateur, ouvrez `http://localhost:9001`.

Connectez-vous avec :

- **Username :** `anfa-admin`
- **Password :** `anfa-password-2026`

Vous arrivez sur la console MinIO. Elle est vide (nouveau déploiement).

Point de vérification 5.4. Vous êtes connecté à la console MinIO qui tourne dans Kubernetes. **Faites une capture d'écran de la console** elle ira dans votre RENDU.

Partie 6 : Observer le self-healing

C'est le **moment-clé pédagogique** de cette séance.

6.1 Lister les pods et noter le nom du pod MinIO

```
kubectl get pods
```

Notez le nom complet, par exemple `minio-6f8d9c7b5-x2k9p`.

6.2 Supprimer le pod brutalement

```
# Remplacez le nom par celui de VOTRE pod  
kubectl delete pod minio-6f8d9c7b5-x2k9p
```

Immédiatement après, lancez la commande de surveillance :

```
kubectl get pods -w
```

Observation. Vous voyez :

1. L'ancien pod passe en `Terminating`.
2. **Quelques secondes plus tard**, un **nouveau pod apparaît** avec un nouveau suffixe aléatoire.
3. Le nouveau pod passe par `Pending` → `ContainerCreating` → `Running`.

Quittez avec Ctrl+C. Listez à nouveau les pods :

```
kubectl get pods
```

Vous voyez **un seul pod, différent de celui que vous aviez supprimé.**

À comprendre. Vous n'avez rien fait pour le faire revenir. C'est le **Deployment** qui a remarqué qu'il manquait un pod (état souhaité = 1 replica, état observé = 0) et qui en a recréé un automatiquement. **C'est le self-healing.**

Si le pod était tombé à 3 h du matin pendant que vous dormiez, Kubernetes aurait fait la même chose.

6.3 Vérifier que les données ont survécu

Le PersistentVolumeClaim est attaché au nouveau pod. Si vous aviez créé un bucket et déposé des fichiers, ils seraient toujours là.

(Pour ce TP, on n'a pas encore mis de données. C'est juste pour comprendre le principe.)

Point de vérification 6. Vous avez supprimé un pod manuellement, et Kubernetes en a créé un nouveau automatiquement. **Faites une capture de** `kubectl get pods` **montrant le pod recréé**, elle ira dans votre RENDU.

Partie 7 : Scaler le Deployment

7.1 Passer à 3 replicas en une commande

```
# Demande à K8s d'avoir 3 copies du pod minio
kubectl scale deployment minio --replicas=3
```

Résultat attendu : `deployment.apps/minio scaled.`

Observer la création des nouveaux pods :

```
kubectl get pods -w
```

Vous voyez **deux nouveaux pods** apparaître et passer en `Running`. Quittez avec Ctrl+C.

```
kubectl get pods
```

Résultat attendu : 3 pods MinIO, tous `Running`.

```
kubectl get deployment minio
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
minio	3/3	3	3	10m

Note importante. En production réelle, MinIO en mode distribué demande une configuration plus complexe (mode `--standalone` n'accepte qu'un seul pod par PVC). Notre test ici vise à observer le **mécanisme de scaling de Kubernetes**, pas la configuration distribuée de MinIO. Les pods supplémentaires partagent le même PVC, ce qui n'est pas idéal en pratique mais la mécanique K8s est bien démontrée.

7.2 Redescendre à 1 replica

```
kubectl scale deployment minio --replicas=1
```

Observez les deux pods supplémentaires passer en `Terminating` puis disparaître :

```
kubectl get pods -w
```

Point de vérification 7. Vous avez scalé de 1 à 3 puis de 3 à 1. **Faites une capture de** `kubectl get pods` **montrant les 3 replicas avant redescente** elle ira dans votre RENDU.

Partie 8 - Aperçu : activer l'Ingress Controller

Cette partie est un **aperçu sans configuration complète**. L'objectif est de comprendre ce qu'est un Ingress Controller, sans entrer dans la configuration HTTP (hors périmètre).

8.1 Installer l'Ingress Controller nginx pour Kind

```
# Applique le manifeste officiel nginx ingress adapté pour Kind
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml
```

Cette commande déploie l'**Ingress Controller** comme un ensemble de pods dans le namespace `ingress-nginx`.

Attendre qu'il soit prêt :

```
# Attendre que le pod du controller passe en Ready (max 300 secondes)
kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=300s
```

Si la commande affiche `error: timed out waiting for the condition` : ce n'est pas une vraie erreur. Le pod télécharge encore son image (le téléchargement peut dépasser 1-2 minutes sur une connexion lente). Vérifiez avec `kubectl get pods -n ingress-nginx` ; tant que le statut est `ContainerCreating`, relancez simplement la commande `kubectl wait` ci-dessus.

Vérifier :

```
kubectl get pods -n ingress-nginx
```

Vous devriez voir un pod `ingress-nginx-controller-...` en `Running` (et éventuellement deux pods `Completed` qui sont des jobs ponctuels).

À comprendre. L'Ingress Controller est lui-même **un pod Kubernetes**. C'est la beauté de K8s : tout est un pod, y compris les composants d'infrastructure.

Dans un vrai déploiement, on définirait ensuite des ressources `Ingress` (un manifeste YAML supplémentaire) qui diraient à ce controller : « route les requêtes vers `anfa.lome/console` vers le Service MinIO console ». On s'arrête ici pour cette séance.

Point de vérification 8. Le pod `ingress-nginx-controller` est en `Running` dans le namespace `ingress-nginx`.

Partie 9 : Nettoyer (optionnel mais recommandé)

Si vous voulez libérer les ressources de votre machine après le TP :

```
# Supprimer toutes les ressources du namespace anfa
kubectl delete namespace anfa

# Détruire entièrement le cluster Kind
kind delete cluster --name anfa
```

Vous pourrez le recréer plus tard avec `kind create cluster --name anfa`.

Pour la séance suivante, vous repartirez d'un cluster propre.

Partie 10 : Rédiger le RENDU et soumettre

10.1 Préparer le RENDU.md

Créez `seance-03/RENDU.md` à partir de ce gabarit :

```
# Rendu Séance 3

**Nom et prénom :** <Votre nom complet>
**Identifiant GitHub :** <votre-username>
**Date de soumission :** <JJ/MM/AAAA>

## Résumé de la séance

<2-4 lignes : Kind installé, cluster Kubernetes créé, namespace anfa configuré, MinIO déployé via 3 manifestes YAML, self-healing observé, scaling testé, Ingress Controller activé.>

## Étapes principales

1. Installation de Kind et kubectl, création du cluster `anfa`.
2. Création du namespace `anfa` et configuration de kubectl.
3. Déploiement de MinIO via 3 manifestes YAML (PVC, Deployment, Service).
4. Observation du self-healing après suppression manuelle d'un pod.
5. Scaling du Deployment de 1 à 3 replicas, puis retour à 1.
6. Activation de l'Ingress Controller nginx.

## Captures d'écran

### Console MinIO accessible via port-forward
![Console MinIO](captures/console-minio.png)

### Self-healing observé
![Pod recréé](captures/self-healing.png)

### Scaling à 3 replicas
![3 replicas MinIO](captures/scaling-3-replicas.png)

## Réponses aux exercices d'application

<À compléter d'après les énoncés fournis avec l'assignment.>

## Difficultés rencontrées

<Aucune | Décrivez brièvement.>
```

10.2 Créer le dossier `captures/`

```
# Toujours dans seance-03/  
mkdir captures
```

Déposez vos trois captures (`console-minio.png`, `self-healing.png`, `scaling-3-replicas.png`).

10.3 Commiter et pousser

```
# Depuis seance-03/, remonter à la racine du dépôt
cd ..

# Vérifier ce qui sera committé
git status

# Ajouter le dossier de la séance
git add seance-03/

# Commit
git commit -m "Séance 3 : Déploiement de MinIO sur Kubernetes avec Kind"

# Pousser la branche
git push -u origin seance-03
```

10.4 Soumettre dans Google Classroom

Soumettez l'URL :

```
https://github.com/<votre-username>/<nom-de-votre-fork>/tree/seance-03
```

Point de vérification final. Votre branche `seance-03` contient les 3 manifestes, le RENDU, et les captures. Vous avez soumis l'URL dans Google Classroom.

Contenu attendu de votre rendu

À la fin de cette séance, votre fork sur la branche `seance-03` doit avoir cette structure :

```
<nom-de-votre-fork>/  (branche seance-03)
├── README.md           (inchangé)
├── .gitignore          (inchangé)
├── .dockerignore       (inchangé)
├── data/               (inchangé, fourni)
├── seance-03/          (créé par vous)
│   ├── RENDU.md
│   ├── manifests/
│   │   ├── minio-pvc.yaml
│   │   ├── minio-deployment.yaml
│   │   └── minio-service.yaml
│   └── captures/
│       ├── console-minio.png
│       ├── self-healing.png
│       └── scaling-3-replicas.png
```

Pièges fréquents et solutions

Symptôme	Cause probable	Solution
<code>kind create cluster</code> échoue : "Cannot connect to the Docker daemon"	Docker Desktop n'est pas démarré	Lancer Docker Desktop et attendre qu'il soit prêt.
<code>kubectl</code> ne trouve pas le cluster (The connection to the server localhost:8080 was refused)	Le contexte kubectl n'est pas configuré	Vérifier avec <code>kubectl config get-contexts</code> ; basculer avec <code>kubectl config use-context kind-anfa</code> .
Le pod MinIO reste en Pending	Souvent : pas assez de ressources allouées à Docker	Augmenter la RAM/CPU dans les paramètres de Docker Desktop.
Le pod MinIO est en CrashLoopBackOff	Erreur dans le manifeste (mauvais args, image inexistante)	<code>kubectl logs deployment/minio</code> pour voir l'erreur.
<code>kubectl port-forward</code> ne fonctionne plus après quelques minutes	Connexion fermée par inactivité	Relancer la commande. C'est normal en dev.

Symptôme	Cause probable	Solution
Le PVC reste en <code>Pending</code>	Aucun StorageClass disponible	Kind fournit un StorageClass <code>standard</code> par défaut. Vérifier avec <code>kubectl get storageclass</code> .
<code>kubectl wait</code> sur l'Ingress Controller affiche <code>timed out waiting for the condition</code>	L'image du controller est encore en cours de téléchargement (lent au premier coup)	Pas une vraie panne. <code>kubectl get pods -n ingress-nginx</code> ; si <code>ContainerCreating</code> , relancer la commande <code>kubectl wait</code> .
<code>kind create cluster</code> échoue : image <code>kindest/node:v1.35.1</code> introuvable	Version de Kind trop ancienne	Mettre à jour Kind ($\geq$ v0.32.0). Vérifier avec <code>kind version</code> .

Pour aller plus loin (optionnel, hors évaluation)

- Déployez votre image `anfa-analyse` (construite en séance 2) dans le cluster Kubernetes. Vous devrez d'abord la charger dans Kind avec `kind load docker-image anfa-analyse:v1 --name anfa`, puis écrire un manifeste Deployment ou Job.
- Créez un **cluster multi-nœuds** avec Kind (1 control plane + 2 workers) en passant un fichier de configuration à `kind create cluster --config config.yaml`. Observez comment les pods sont répartis sur les nœuds.
- Explorez `kubectl describe pod <nom>` : c'est la commande de diagnostic la plus utile au quotidien.
- Installez le **Kubernetes Dashboard** (interface web officielle) et explorez votre cluster visuellement.

Fin du TP de la séance 3.

Vous savez désormais déployer une application sur Kubernetes, observer son self-healing, et la scaler. La séance 4 vous fera franchir une étape supplémentaire : décrire toute cette infrastructure (le cluster et ses ressources) sous forme de **code versionné** avec Terraform l'**Infrastructure as Code**.